

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №6
по дисциплине «Нейронные сети»
Тема: Прогноз успеха фильмов по обзорам

Студент гр. 1306

Шаврин А.П.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы.

Прогноз успеха фильмов по обзорам (Predict Sentiment From Movie Reviews)

Задание.

- Ознакомиться с задачей классификации
- Изучить способы представления текста для передачи в ИНС
- Достигнуть точность прогноза не менее 95%

Выполнение работы.

Исследование данных

Проведен анализ структуры датасета:

- Категории: [0 1] (отрицательные и положительные отзывы).
- Количество уникальных слов в выборке при num_words=10000 составило 9998 (остальные два — служебные токены).
- Статистика длины рецензий: средняя длина – 235 слов, стандартное отклонение – 173, максимальная – 2494 слова.п

В индексации слов предусмотрены специальные токены:

0 – padding (дополнение до одинаковой длины)

1 – start (начало текста)

2 – unknown (неизвестное слово)

3 – unused (зарезервирован)

При декодировании индексов в слова используется сдвиг на 3, поэтому, например, индекс 1 соответствует слову the (после вычитания 3 получаем 4, что в исходном словаре действительно соответствует слову the). Это стандартная практика в наборе данных IMDb.

Предобработка данных

Для подачи в нейронную сеть исследованы 2 способа преобразования текста:

- One-hot векторизация – каждый отзыв представляется двоичным вектором длины, равной размеру словаря, где единицами отмечены присутствующие слова.
- Padding + Embedding – отзывы дополняются нулями до фиксированной длины (`max_len=300`) и затем пропускаются через слой Embedding, который преобразует индексы в плотные векторные представления. В качестве максимальной длины (`max_len`) было выбрано значение 300. Это решение основано на анализе статистики длин рецензий в датасете. Средняя длина отзыва составляет 235 слов. Стандартное отклонение – 173. 75% отзывов имеют длину не более 285 слов. Выбор 300 слов позволяет сохранить практически полный текст для подавляющего большинства рецензий (около 85–90% отзывов укладываются в этот лимит). Более длинные отзывы (редкие) обрезаются, что может приводить к незначительной потере информации, но это оправдано, так как основные смысловые обороты обычно сосредоточены в начале или середине текста. Если бы мы выбрали слишком маленькое значение (например, 100), многие отзывы были бы сильно урезаны, и модель теряла бы важный контекст. Если бы было выбрано слишком большое значение (например, 1000), это увеличило бы размерность входных данных, замедлило обучение и могло бы привести к переобучению, так как длинные отзывы – редки. Таким образом, значение 300 является компромиссом, учитывающим распределение длин и эффективность обучения.

Построение и обучение моделей

1. Модель на основе one-hot представления.

Использована полносвязная сеть со следующей архитектурой:

- Входной слой: размерность = размер словаря
- Скрытые слои: Dense(64, relu), Dropout(0.3), Dense(64, relu), Dropout(0.3), Dense(32, relu)
- Выходной слой: Dense(1, sigmoid)

Обучение проводилось с оптимизатором adam, функцией потерь binary_crossentropy, метрикой accuracy. Количество эпох – 2-3, размер батча – 500–512.

2. Модель на основе Embedding

Архитектура:

- Embedding(input_dim=размер словаря, output_dim=128, input_length=300) - преобразует каждый целочисленный индекс слова в плотный вектор фиксированной размерности (output_dim=128).
- GlobalAveragePooling1D() - вычисляет среднее значение каждого из 128 признаков по всей длине последовательности, чтобы передать эти данные в полносвязные слои
- Dense(128, relu), Dropout(0.4)
- Dense(64, relu)
- Dense(1, sigmoid)

Обучение проводилось в течение 6 эпох, батч 512.

Эксперименты.

Проведены эксперименты с размерами словаря 2000, 5000, 10 000, 15 000 при one-hot кодировании. Результаты в ходе обучения представлены на рисунках 1-4, а результаты на тестовой выборке представлены в таблице 1.

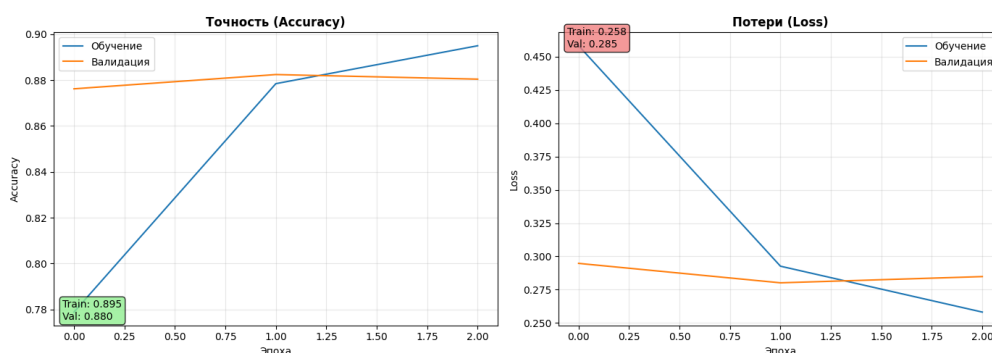


Рисунок 1. Точность и потери при one-hot кодировании с размером словаря 2000.

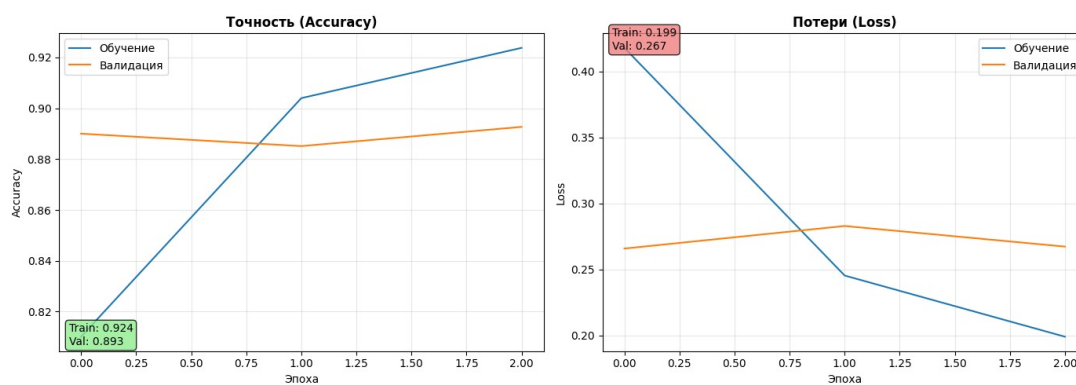


Рисунок 2. Точность и потери при one-hot кодировании с размером словаря 5000.

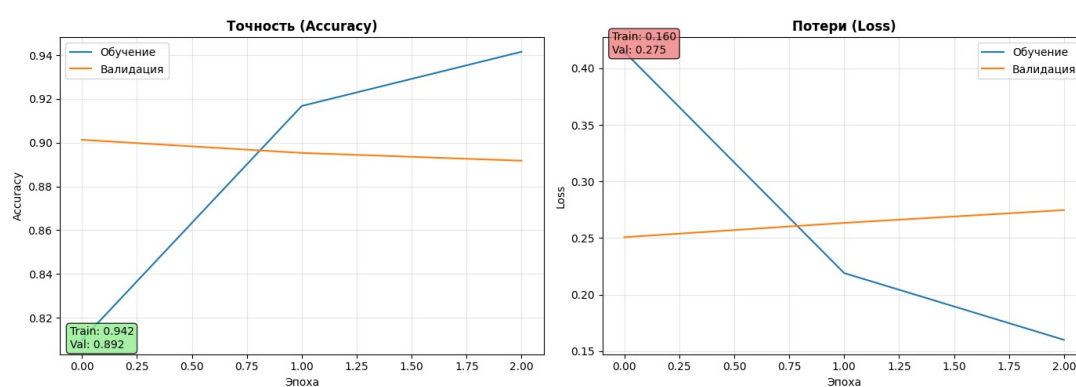


Рисунок 3. Точность и потери при one-hot кодировании с размером словаря 10000.

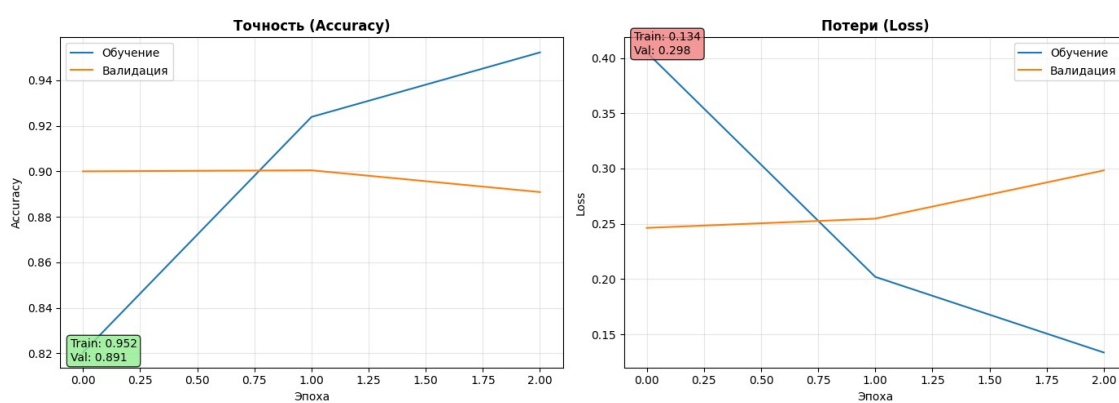


Рисунок 4. Точность и потери при one-hot кодировании с размером словаря 15000.

Таблица 1. Тестовые точность и потери при one-hot кодировании.

Размер словаря	Тестовая точность	Тестовые потери
2000	0.8736	0.2973
5000	0.8900	0.2699
10 000	0.8926	0.2799
15 000	0.8892	0.3059

Анализируя результаты экспериментов можно заметить, что в экспериментах с one-hot представлением текста размер словаря напрямую определяет размерность входного вектора (количество признаков). Каждое слово кодируется битом в этом векторе, поэтому чем больше словарь, тем больше потенциально информативных признаков получает модель.

При словаре в 2000 слов модель оперирует только самыми частотными словами. Многие важные, но менее частотные термины (например, названия актеров, специфическая лексика) исключаются. Это приводит к потере части смысла и ограничивает точность (0.8736).

Увеличение словаря до 5000 и 10000 добавляет в модель все больше значимых слов. Модель начинает лучше различать положительные и отрицательные отзывы, поскольку получает доступ к более полному набору признаков. Наивысшая точность достигается при 10 000 слов (0.8926).

При расширении словаря до 15 000 в модель добавляются редкие слова, которые встречаются всего в нескольких рецензиях. Модель начинает запоминать эти редкие признаки как уникальные идентификаторы конкретных примеров, вместо того чтобы обобщать закономерности. Это классический признак переобучения: точность на обучающей выборке растет (до 0.9523), а на тестовой – падает (0.8892). Кроме того, увеличение входной размерности (15000 против 10000) добавляет 5000 новых параметров, что при фиксированном объеме данных также способствует переобучению.

Проведены эксперименты с размерами словаря 2000, 5000, 10 000, 15 000 при embedding. Результаты в ходе обучения представлены на рисунках 5-8, а результаты на тестовой выборке представлены в таблице 2.

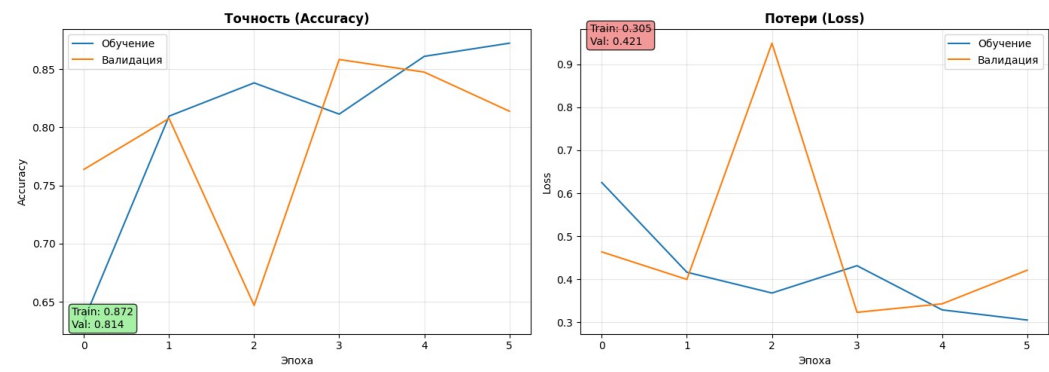


Рисунок 5. Точность и потери при embedding с размером словаря 2000.

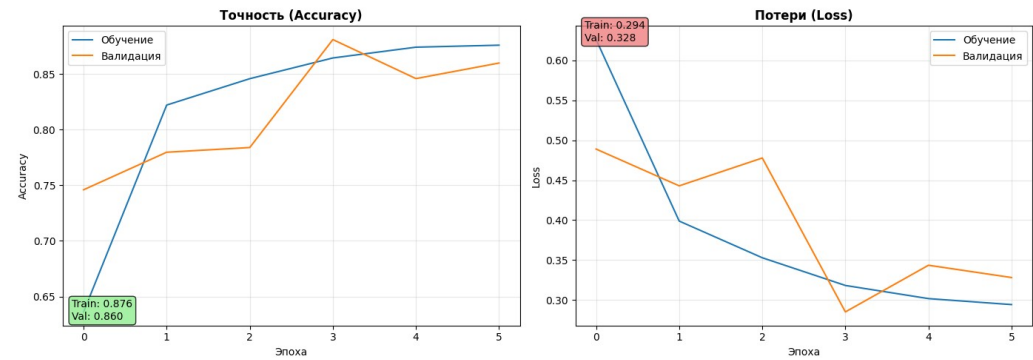


Рисунок 6. Точность и потери при embedding с размером словаря 5000.

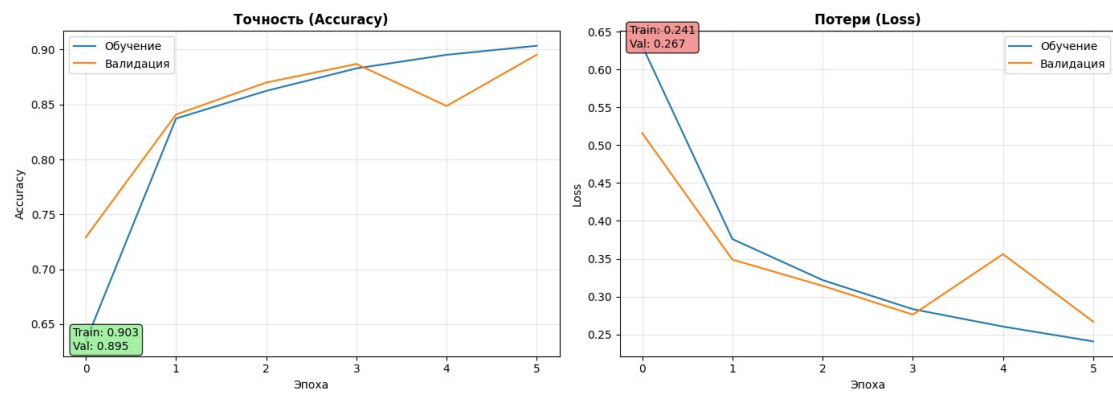


Рисунок 7. Точность и потери при embedding с размером словаря 10000.

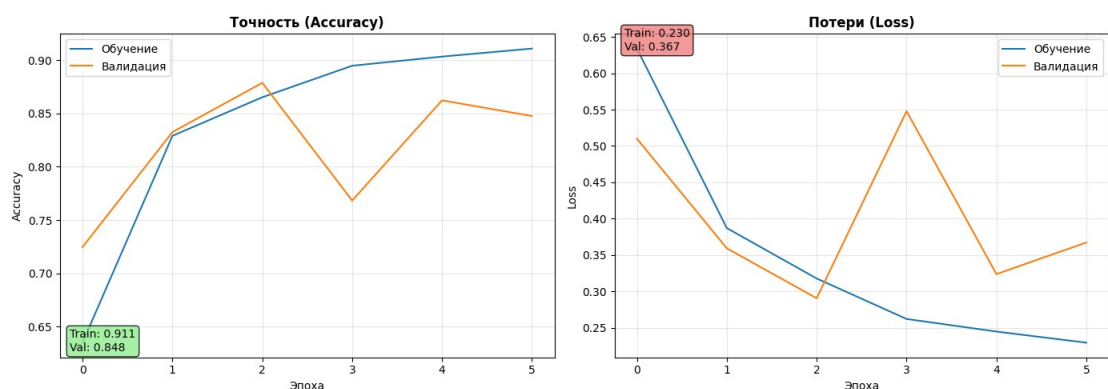


Рисунок 8. Точность и потери при embedding с размером словаря 15000.

Таблица 2. Тестовые точность и потери при embedding.

Размер словаря	Тестовая точность	Тестовые потери
2000	0.8138	0.4286
5000	0.8622	0.3388
10 000	0.8920	0.2794
15 000	0.8464	0.3706

Анализируя результаты исследования для моделей с embedding-слоем размер словаря определяет количество обучаемых векторов в первом слое. Каждый вектор имеет размерность 128, поэтому общее число параметров слоя Embedding равно $\text{input_dim} * 128$.

При словаре в 2000 слов количество обучаемых параметров относительно невелико (256 000), но словарь слишком мал, чтобы охватить ключевую лексику. Точность на тесте составляет всего 0.8138, модель не может построить качественные семантические представления.

Словарь 5000 увеличивает количество параметров до 640 000. Точность растет до 0.8622, модель начинает улавливать больше смысловых нюансов.

При 10 000 слов достигается наилучшая точность 0.8920. Словарь уже содержит большинство значимых слов, а количество параметров (1 280 000) еще не приводит к переобучению.

Словарь 15 000 добавляет 5 000 новых векторов (640 000 дополнительных параметров). Редкие слова, входящие в этот диапазон, имеют слишком мало примеров в обучающей выборке. Их эмбединги плохо обучаются и вносят шум. Как следствие, точность падает до 0.8464, а потери растут (0.3706). Это также свидетельствует о переобучении: модель начинает подстраиваться под редкие шумовые признаки, теряя способность к обобщению.

Работа на пользовательском тексте

Была написана функция, которая анализирует пользовательский текст по предоставленной модели (рисунок 9)

```
[55] ✓ 0s
def predict_text(model, text, word_index, dimension=10000):
    vector = encode_text(text, word_index, dimension)

    prediction = model.predict(vector)[0][0]

    print("Текст:", text)
    print("Вероятность положительного отзыва:", prediction)

    if prediction > 0.5:
        print("Результат: ПОЛОЖИТЕЛЬНЫЙ")
    else:
        print("Результат: ОТРИЦАТЕЛЬНЫЙ")
```

Рисунок 9. Функция, которая анализирует пользовательский текст по предоставленной модели.

Для демонстрации обучена модель (one-hot, размер словаря 15 000) и протестирована на нескольких пользовательских фразах. Результаты представлены на рисунке 10.

```
1/1 ————— 0s 360ms/step
Текст: This movie is great amazing wonderful fanta
Вероятность положительного отзыва: 0.7660979
Результат: ПОЛОЖИТЕЛЬНЫЙ
1/1 ————— 1s 617ms/step
Текст: This movie is awful
Вероятность положительного отзыва: 0.29600632
Результат: ОТРИЦАТЕЛЬНЫЙ
1/1 ————— 0s 149ms/step
Текст: The movie was okay
Вероятность положительного отзыва: 0.5456581
Результат: ПОЛОЖИТЕЛЬНЫЙ
1/1 ————— 0s 118ms/step
Текст: The acting was bad but the story was good
Вероятность положительного отзыва: 0.42340833
Результат: ОТРИЦАТЕЛЬНЫЙ
```

Рисунок 10. Пользовательские отзывы.

Модель правильно распознает явно положительные и отрицательные отзывы. Нейтральный отзыв ("okay") классифицирован как положительный (вероятность чуть выше 0.5), а противоречивый отзыв – как отрицательный, что может быть обусловлено перевесом негативных слов.

Анализ достигнутых результатов

Максимальная точность, полученная в ходе экспериментов, составила 0.8926 (one-hot, размер словаря 10 000). Это значение не достигает заданной планки 95%, что объясняется следующими факторами:

- Простота архитектуры – полносвязные сети не учитывают порядок слов, что важно для понимания контекста.
- Объем обучающих данных – хотя использовалось 45 000 примеров (после разделения 50 000 на обучение и валидацию), этого недостаточно для обучения глубоких моделей с большим числом параметров без риска переобучения.
- Ограничения вычислительных ресурсов – для one-hot представления при размере словаря 15 000 входной вектор имел размерность 15 000, что при 45 000 примерах требовало значительный объем оперативной памяти только для хранения признаков. Попытка использовать словарь больше 15 000 приводила к исчерпанию памяти в Google Colab, что ограничило эксперименты этим значением.
- Отсутствие тонкой настройки гиперпараметров – количество эпох, размер слоев, dropout подбирались интуитивно, не проводился систематический поиск.
- Переобучение – на модели с embedding при словаре 15 000 точность на обучении достигала 0.911, а на тесте 0.846, что свидетельствует о подстройке под редкие шумовые признаки. Аналогично, one-hot модель при словаре 15 000 показывала на обучении 0.952, а на тесте 0.889.

Для достижения 95% и выше необходимо использовать более сложные архитектуры, например, рекуррентные сети (LSTM, GRU).

Выводы.

В ходе лабораторной работы были реализованы и обучены две нейросетевые модели для классификации тональности отзывов на основе one-hot представления и embedding. Проведено исследование влияния размера словаря на качество классификации, по результатам которого оптимальным оказался словарь из 10 000 слов. Показано, что увеличение размера словаря свыше 10 000 не приводит к росту точности из-за переобучения и шума от редких слов. Достигнутая точность (~89%) соответствует результатам, полученным создателями датасета (88.89%). Продемонстрирована работа модели на пользовательских текстах.

Таким образом, задача классификации сентимента решена с качеством, близким к стандартам на момент создания датасета, но не достигнута планка в 95% из-за ограничений использованных архитектур и вычислительных ресурсов.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lab6.ipynb

```
"""# Установка зависимостей
"""

!pip install pandas numpy matplotlib scikit-learn tensorflow

"""# Импорт"""

import numpy as np
from keras.datasets import imdb
from keras import models, layers
import matplotlib.pyplot as plt
import pandas as pd
from keras.preprocessing.sequence import pad_sequences
from keras import regularizers

"""# Загрузка данных"""

def load_data(num_words=10000):
    (train_data, train_targets), (test_data, test_targets) =
imdb.load_data(num_words=num_words)

    data = np.concatenate((train_data, test_data), axis=0)
    targets = np.concatenate((train_targets, test_targets), axis=0)

    return data, targets

data, targets = load_data(num_words=10000)
word_index = imdb.get_word_index()

"""# Анализ данных"""

print("Категории:", np.unique(targets))
print("Количество уникальных слов:", len(np.unique(np.hstack(data))), end=3*\n')

df = pd.DataFrame({
    'review_length': [len(review) for review in data],
    'sentiment': targets
})

print(df['review_length'].describe(), end=3*\n')
print(df.groupby('sentiment')['review_length'].describe())

print("Label:", targets[0])
print(data[0])

print(word_index)
print(len(word_index))
```

```

reverse_index = {value: key for key, value in word_index.items()}
dict(sorted(reverse_index.items()))

"""**Служебные токены:**
* 0 - padding (выравнивание длины)
* 1 - start (начало текста)
* 2 - unknown (неизвестное слово)
* 3 - unused (зарезервировано)

"""

def decode_review(encoded_review, word_index):
    reverse_index = {value: key for key, value in word_index.items()}
    return " ".join([reverse_index.get(i - 3, "#") for i in encoded_review])

decode_review(data[0], word_index)

"""# Предобработка данных"""

def split_data(data, targets, test_size=5000):
    x_test = data[:test_size]
    y_test = targets[:test_size]

    x_train = data[test_size:]
    y_train = targets[test_size:]

    return x_train, y_train, x_test, y_test

def vectorize_sequences(sequences, dimension=10000):
    result = np.zeros((len(sequences), dimension), dtype=np.float32)
    for i, sequence in enumerate(sequences):
        result[i, sequence] = 1.0
    return result

def padding_sequences(sequences, max_len=250):
    return pad_sequences(
        sequences,
        maxlen=max_len,
        padding='post',
        truncating='post'
    )

data = vectorize_sequences(data)
targets = np.array(targets).astype("float32")

x_train, y_train, x_test, y_test = split_data(data, targets)

```

```

"""# Создание модели"""

def build_model(input_dim):
    model = models.Sequential()

    model.add(layers.Dense(64, activation="relu", input_shape=(input_dim,)))
    model.add(layers.Dropout(0.3))

    model.add(layers.Dense(64, activation="relu"))
    model.add(layers.Dropout(0.3))

    model.add(layers.Dense(32, activation="relu"))

    model.add(layers.Dense(1, activation="sigmoid"))

    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )

    return model

def build_embedding_model(num_words, max_len=250):
    model = models.Sequential()

    model.add(layers.Embedding(
        input_dim=num_words,
        output_dim=128,
        input_length=max_len
    ))

    model.add(layers.GlobalAveragePooling1D())

    model.add(layers.Dense(128, activation="relu"))
    model.add(layers.Dropout(0.4))

    model.add(layers.Dense(64, activation="relu"))

    model.add(layers.Dense(1, activation="sigmoid"))

    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )

    return model

def plot_training_history(history):
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

```

```

# График точности
axes[0].plot(history.history['accuracy'], label='Обучение')
axes[0].plot(history.history['val_accuracy'], label='Валидация')

axes[0].set_title('Точность (Accuracy)', fontsize=12, fontweight='bold')
axes[0].set_xlabel('Эпоха')
axes[0].set_ylabel('Accuracy')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[0].text(0.02, 0.02,
            f"Train: {history.history['accuracy'][-1]:.3f}\n"
            f"Val: {history.history['val_accuracy'][-1]:.3f}",
            transform=axes[0].transAxes,
            bbox=dict(boxstyle='round', facecolor='lightgreen', alpha=0.8))

# График потерь
axes[1].plot(history.history['loss'], label='Обучение')
axes[1].plot(history.history['val_loss'], label='Валидация')

axes[1].set_title('Потери (Loss)', fontsize=12, fontweight='bold')
axes[1].set_xlabel('Эпоха')
axes[1].set_ylabel('Loss')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

axes[1].text(0.02, 0.95,
            f"Train: {history.history['loss'][-1]:.3f}\n"
            f"Val: {history.history['val_loss'][-1]:.3f}",
            transform=axes[1].transAxes,
            bbox=dict(boxstyle='round', facecolor='lightcoral', alpha=0.8))

plt.tight_layout()
plt.show()

"""# Обучение модели"""

model = build_model(10000)
model.summary()

history = model.fit(
    x_train,
    y_train,
    epochs=2,
    batch_size=500,
    validation_split=0.1,
    verbose=1
)
test_loss, test_acc = model.evaluate(x_test, y_test)

print(f"Тестовая точность: {test_acc:.4f}")

```

```

print(f"Тестовые потери: {test_loss:.4f}")

plot_training_history(history)

"""# Исследование"""

def run_experiment(num_words_list):
    results = {}

    for dim in num_words_list:
        print(f"Размер словаря: {dim}")

        data, targets = load_data(num_words=dim)
        data = vectorize_sequences(data, dimension=dim)
        targets = np.array(targets).astype("float32")
        x_train, y_train, x_test, y_test = split_data(data, targets)

        model = build_model(dim)
        history = model.fit(
            x_train, y_train,
            epochs=3,
            batch_size=512,
            validation_split=0.1,
            verbose=1
        )

        test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
        print(f"Точность на тесте: {test_acc:.4f}, Потери: {test_loss:.4f}")

        results[dim] = {
            "history": history,
            "test_acc": test_acc,
            "test_loss": test_loss
        }

    return results

def run_embedding_experiment(num_words_list, max_len=250):
    results = {}

    for dim in num_words_list:
        print(f"Размер словаря: {dim}")

        data, targets = load_data(num_words=dim)
        data = padding_sequences(data, max_len=max_len)
        targets = np.array(targets).astype("float32")
        x_train, y_train, x_test, y_test = split_data(data, targets)

        model = build_embedding_model(dim, max_len)

        history = model.fit(
            x_train, y_train,

```



```

        epochs=6,
        batch_size=512,
        validation_split=0.1,
        verbose=1
    )

    test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)

    print(f"Точность на тесте: {test_acc:.4f}, Потери: {test_loss:.4f}")

    results[dim] = {
        "history": history,
        "test_acc": test_acc,
        "test_loss": test_loss
    }

    return results

num_words_list = [2000, 5000, 10000, 15000]
results = run_experiment(num_words_list)

for dim in num_words_list:
    plot_training_history(results[dim]["history"])

num_words_list = [2000, 5000, 10000, 15000]
results = run_embedding_experiment(num_words_list, max_len=300)

for dim in num_words_list:
    plot_training_history(results[dim]["history"])

""" # Пользовательский текст"""

def encode_text(text, word_index, dimension):
    words = text.lower().split()
    encoded = []

    for word in words:
        index = word_index.get(word)
        if index is not None and index < dimension:
            encoded.append(index)

    vector = np.zeros((1, dimension))
    vector[0, encoded] = 1.0
    return vector

def predict_text(model, text, word_index, dimension=10000):
    vector = encode_text(text, word_index, dimension)

    prediction = model.predict(vector)[0][0]

    print("Текст:", text)
    print("Вероятность положительного отзыва:", prediction)

```

```

if prediction > 0.5:
    print("Результат: ПОЛОЖИТЕЛЬНЫЙ")
else:
    print("Результат: ОТРИЦАТЕЛЬНЫЙ")

data, targets = load_data(num_words=15000)
data = vectorize_sequences(data, dimension=15000)
targets = np.array(targets).astype("float32")
x_train, y_train, x_test, y_test = split_data(data, targets)

user_text_model = build_model(dim)
history = user_text_model.fit(
    x_train, y_train,
    epochs=3,
    batch_size=512,
    validation_split=0.1,
    verbose=0
)

test_loss, test_acc = user_text_model.evaluate(x_test, y_test, verbose=0)

print(f"Точность на тесте: {test_acc:.4f}, Потери: {test_loss:.4f}")

text = input()
predict_text(user_text_model, text, word_index, dimension=15000)

print(word_index["terrible"])
print(word_index["awful"])
print(word_index["borin"])
print(word_index["great"])
print(word_index["amazing"])
print(word_index["wonderful"])

# Хороший отзыв
text1 = "This movie is great amazing wonderful fantastic"
predict_text(user_text_model, text1, word_index, dimension=15000)

# Плохой отзыв
text2 = "This movie is awful"
predict_text(user_text_model, text2, word_index, dimension=15000)

# Нейтральный отзыв
text3 = "The movie was okay"
predict_text(user_text_model, text3, word_index, dimension=15000)

# Противоречивый отзыв
text6 = "The acting was bad but the story was good"
predict_text(user_text_model, text6, word_index, dimension=15000)

```